IOT PRACTICUM JOURNAL

MODULE 4

LONG RANGE COMMUNICATION (LoRa P2P & LoRaWAN)

Group : .....

Day/Shift : .....

Instructor Assistant Code : .....

Name : .....

NIM : .....

IOT DEVICE PROGRAMMING PRACTICUM

SCHOOL OF ELECTRICAL ENGINEERING

TELKOM UNIVERSITY

2026

A. Circuit and System

This module comes in **two parts**. **Part A** uses peer-to-peer LoRa in the form of a game: one Beacon board is hidden, and every group hunts it by listening to how strong its signal is. **Part B** uses LoRaWAN: the same board talks to a WM1302 gateway through a protocol that manages the channels itself. Comparing the two is the whole point of this module — in Part A you build every network rule yourselves, while in Part B the protocol has already supplied them.

A.A. Part A — The LoRa Hunt

How the game works. An assistant hides **one** Beacon board somewhere within range of the hotspot. That beacon transmits continuously, five times a second. Each group holds one Hunter board that **only listens**, then reports the signal strength it catches to the scoreboard over Wi-Fi. The closer you are, the stronger the reading. The first group actually holding the Beacon board wins.

1. Prepare the following components:

   - Cosmic LoRa Aurora V2 (1x per group, as the Hunter)
   - Cosmic LoRa Aurora V2 (1x per class, as the Beacon)
   - 920 MHz LoRa Antenna (1x per board)
   - Orange Pi 4 PRO (1x per class, hotspot and scoreboard)
   - Power bank or a long USB cable (1x per group)

2. Unlike the previous modules, Part A uses **no external sensor or actuator at all**: what is being measured is the radio wave itself. There is therefore nothing to wire — fit the antenna and power the board up. The table below records the radio pins already wired inside the board and used by the program.

Module 4 Part A LoRa Hunt topology diagram (to follow)

RFM95W radio pins on the Cosmic LoRa Aurora V2			
Signal	**Pin**	**Signal**	**Pin**
RFM (NSS)	GPIO15	RFM (DIO0)	GPIO27
RFM (RST)	GPIO0	RFM (SCK)	GPIO18
RFM (MISO)	GPIO19	RFM (MOSI)	GPIO23
RFM (EN)	GPIO32		

*Note: fit the 920 MHz antenna **before** the USB cable goes in. Transmitting without an antenna can damage the radio permanently, and that applies to the Beacon board as much as to a Hunter. GPIO 15, 0, 27, 32, 18,*

19 and 23 are already taken by the radio module and must not be used for anything else. A Hunter needs no voltage divider because nothing is wired to it.

File assignment per group. Every group has its own folder with its group number already in it. Copy your group's folder and upload `hunter.ino` to your board — there is no line to edit. The `beacon.ino` inside that folder is identical for every group and is only uploaded by an assistant, to the Beacon board.

Group	Program folder	Group	Program folder
1	`modul4/grup1/`	5	`modul4/grup5/`
2	`modul4/grup2/`	6	`modul4/grup6/`
3	`modul4/grup3/`	7	`modul4/grup7/`
4	`modul4/grup4/`	–	–

3. Before running the program, prepare your working environment in the following order. Tick every step you have completed!

   - ☐ Fitting the 920 MHz antenna to the board **before** plugging the USB cable in
   - ☐ Installing the `LoRa` library by Sandeep Mistry through the Arduino IDE Library Manager
   - ☐ Installing the `WebSockets` library by Markus Sattler through the Arduino IDE Library Manager
   - ☐ Selecting the ESP32 Dev Module board in the Arduino IDE
   - ☐ Checking that the Orange Pi has its `IRIS-HUNT` hotspot and its scoreboard running
   - ☐ Copying your group's folder, then uploading `hunter.ino` without editing anything
   - ☐ Checking that the Serial Monitor prints an IP address and a synchronised clock
   - ☐ Opening the scoreboard at `http://192.168.4.1:8080`, entering your own student ID, and confirming it is your group that appears
   - ☐ Calibrating the distance reference: hold one Hunter exactly 1 metre from the Beacon, then press **Calibrate 1 m** on its card
   - ☐ Turning on **Distance ticks** on whichever phone is leading the group, and setting its screen timeout to 5 minutes before setting off

   *Regulatory note: the band permitted in Indonesia is 920–923 MHz (AS923-2), so the frequency used in this practicum is `920E6`. The values `433E6`, `866E6` and `915E6` belong to other regions and **must not** be used here. The beacon transmits at SF7 with 250 kHz bandwidth, so one packet is only about 20 milliseconds in the air; with a 200 millisecond gap the channel is busy roughly a tenth of the time. A rate that dense is only appropriate for a short, low-power practicum session; a permanently installed AS923 device is kept far below it.*

   *Transmit power note: the beacon's power is matched to the **size of the arena**, not turned up as far as it goes. At 20 dBm the whole building reads as equally strong and the beacon becomes impossible to find; but at too low a power the farthest group never hears it at all and the game dies just as surely. The default `BEACON_TX_POWER` is **10 dBm**, which by link budget at SF7 is still audible at roughly a kilometre outdoors and a few hundred metres inside a walled building. Use 2 for a single room, 10 for one floor of a building, and 14 for a large building.*

4. Complete the program below by filling in the blank sections. Afterwards (or while completing it), retype the blank sections you have filled in using the VS Code/Arduino IDE application prepared by your respective lab assistants!

Blank sections are marked with an underscore run `_____` and numbered `(1)` through `(6)` in the comment to their right. Replace each underscore run with the correct code following the hint in its comment. All four radio values — frequency, spreading factor, bandwidth and sync word — must match exactly between Hunter and Beacon; if even one differs, the packets are never heard at all.

Hunter Source Code — LoRa Hunt (Cosmic LoRa Aurora V2)

```
 */

#include <WiFi.h>
#include <SPI.h>
#include <_____>  // (1) LoRa peer-to-peer library header by Sandeep Mistry
#include <WebSocketsClient.h>
#include <time.h>
#include <sys/time.h>
…
void connectToWiFi() {
  Serial.print("Connecting to the Orange Pi hotspot");

  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, _____);  // (2) Wi-Fi password variable

  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
…
    Serial.println("RFM95W radio not detected yet, retrying...");
    delay(500);
  }

  LoRa.setSpreadingFactor(_____);  // (3) Spreading factor constant
  LoRa.setSignalBandwidth(LORA_BANDWIDTH);
  LoRa.setSyncWord(_____);  // (4) Pairing marker constant, same as the beacon
}

void setup() {
  Serial.begin(115200);
…
  }

  String receivedAt = wallClockTimestamp();

  int rssi = LoRa._____();  // (5) Method that reads the packet signal strength
  float snr = LoRa.packetSnr();

  // A beacon packet holds "<packet no.> <send time>", separated by one space.
  int space = beaconLine.indexOf(' ');
…
            GROUP_ID, seq.c_str(), rssi, smoothRssi(rssi), snr,
            sentAt.c_str(), receivedAt.c_str(),
            WiFi.localIP().toString().c_str());

  webSocket._____(payload);  // (6) Method that sends a text message

  Serial.printf("[%s] Packet #%s | RSSI %d dBm (smoothed %.1f) | SNR %.1f dB\n",
                receivedAt.c_str(), seq.c_str(), rssi, smoothed, snr);
}
```

Beacon Source Code — LoRa Hunt (Cosmic LoRa Aurora V2)

```
 */

#include <WiFi.h>
#include <SPI.h>
#include <_____>  // (1) LoRa peer-to-peer library header by Sandeep Mistry
#include <WebSocketsClient.h>
#include <time.h>
#include <sys/time.h>
…
}

```

```
92  void syncClock() {
93    Serial.print("Synchronising the clock to the Orange Pi");
94    configTime(GMT_OFFSET_SEC, DST_OFFSET_SEC, ________);  // (2) NTP server address
95
96    struct tm timeinfo;
97    while (!getLocalTime(&timeinfo, 1000)) {
98      Serial.print(".");
…
114     delay(500);
115   }
116
117   LoRa.setSpreadingFactor(LORA_SPREADING_FACTOR);
118   LoRa.setSignalBandwidth(________);  // (3) Radio bandwidth constant
119   LoRa.setSyncWord(LORA_SYNC_WORD);
120   LoRa.setTxPower(________);  // (4) Beacon transmit power constant
121 }
122
123 void setup() {
124   Serial.begin(115200);
…
148   String sentAt = wallClockTimestamp();
149   String payload = String(seq) + " " + sentAt;
150
151   LoRa.beginPacket();
152   LoRa.print(________);  // (5) Variable holding the packet built above
153   LoRa.________();  // (6) Method that transmits the prepared packet
154
155   // The scoreboard needs to know which packet numbers were actually sent, so
156   // that a packet which never reached a hunter can be counted as lost.
157   char report[64];
```

5. Record the configuration you used in the following table! The first four radio values must match exactly between Hunter and Beacon for any packet to be read.

Parameter	Value used
Operating frequency (`LORA_FREQUENCY`)	
Spreading factor (`LORA_SPREADING_FACTOR`)	
Bandwidth (`LORA_BANDWIDTH`)	
Sync word (`LORA_SYNC_WORD`)	
Beacon transmit power (`BEACON_TX_POWER`)	
Beacon interval (`BEACON_PERIOD_MS`)	
Hotspot name and address (`WIFI_SSID` / `SERVER_HOST`)	
Group number (`GROUP_ID`) and your board's IP address	

A.B. Part B — LoRaWAN through the WM1302 Gateway

6. Prepare the following components:

- Cosmic LoRa Aurora V2 (the Part A board) (1x per group)
- 920 MHz LoRa Antenna (1x per group)
- HC-SR04 Ultrasonic Sensor (1x per group)
- 1 kΩ and 2 kΩ Resistor (1x per group, ECHO divider)
- Breadboard (1x per group)
- Wio-WM1302 gateway + hat (1x per class)
- Orange Pi 4 PRO (ChirpStack) (1x per class)
- Wires/Jumpers (as needed)

7. Unlike Part A, which used no sensor at all, Part B goes back to sending HC-SR04 readings. Wire the sensor as in the table below, and note that the LMIC library also uses one extra radio pin.

HC-SR04 sensor (wired by you)		RFM95W radio (already on the board)	
Component	**Pin**	**Signal**	**Pin**
US (VCC)	5V	RFM (NSS)	GPIO15
US (TRIG)	GPIO16	RFM (RST)	GPIO0
US (ECHO) ⟶ R1K	GPIO17	RFM (DIO0)	GPIO27
R1K ⟶ R2K	GND	RFM (DIO1)	GPIO2
US (GND)	GND	RFM (EN/VMODEM)	GPIO32

Note: the HC-SR04's ECHO pin outputs 5 V, so it must pass through a 1 kΩ/2 kΩ voltage divider before reaching GPIO17; without that divider the ESP32 pin can be destroyed. On the radio side there is only one difference from Part A: peer-to-peer LoRa needs DIO0 alone, while LoRaWAN also needs DIO1 to mark the receive windows that follow a transmission. DIO1 is already wired to GPIO2 on the Cosmic board, so no cable has to be moved.

8. Before running the program, prepare your working environment in the following order. Tick every step you have completed!

   - ☐ Installing the `MCCI LoRaWAN LMIC library` through the Arduino IDE Library Manager
   - ☐ Making sure `lmic_project_config.h` uses the `CFG_as923` band and `CFG_sx1276_radio`
   - ☐ Registering your group's board in ChirpStack with the DEVEUI from the table above
   - ☐ Copying the APPKEY from the ChirpStack device page into the node program
   - ☐ Making sure the WM1302 gateway shows as *online* in ChirpStack
   - ☐ Syncing the node board's clock from the PC: `.\scripts\send_time.ps1 -Port COM_A`
   - ☐ Running the gateway monitor program on the Orange Pi 4 PRO

Note: in Part A you have to hand out transmit slots yourselves so that eight boards do not collide. In Part B no such handout is needed, because the WM1302 gateway listens on eight channels at once and the LoRaWAN protocol decides when each node may transmit. This is exactly why gateways exist.

9. Complete the program below by filling in the blank sections, just as in Part A!

Node Source Code — LoRaWAN OTAA (Cosmic LoRa Aurora V2)

```
 * Fill in every blank; the numbers (1) through (6) in the comment beside each
 * one are the hints.
 */

#include <________>  // (1) Main header of the MCCI LoRaWAN LMIC library
#include <hal/hal.h>
#include <SPI.h>
#include <time.h>
…
// registered separately in ChirpStack. APPKEY is copied from the device
// page in ChirpStack, written in MSB order.
static const u1_t PROGMEM APPEUI[8]  = { 0, 0, 0, 0, 0, 0, 0, 0 };
static const u1_t PROGMEM DEVEUI[8]  = { GROUP_ID, 0x00, 0x00, 0x00, 0x00, 0x00, 0x53,
0x49 };
static const u1_t PROGMEM APPKEY[16] = { ________ };  // (2) The sixteen APPKEY bytes from
ChirpStack

void os_getArtEui(u1_t *buf) { memcpy_P(buf, APPEUI, 8); }
```

```
68  void os_getDevEui(u1_t *buf) { memcpy_P(buf, DEVEUI, 8); }
    void os_getDevKey(u1_t *buf) { memcpy_P(buf, APPKEY, 16); }
71  …
72  const lmic_pinmap lmic_pins = {
73    .nss  = LORA_NSS,
74    .rxtx = LMIC_UNUSED_PIN,
75    .rst  = LORA_RST,
76    .dio  = { LORA_DIO0, ________, LMIC_UNUSED_PIN },  // (3) DIO1 pin constant
77  };
78
    static osjob_t sendjob;
148 …
149   digitalWrite(TRIG_PIN, HIGH);
150   delayMicroseconds(10);
151   digitalWrite(TRIG_PIN, LOW);
152
153   unsigned long duration = ________(ECHO_PIN, HIGH, ECHO_TIMEOUT_US);  // (4) Pulse-width
154 measuring function
155
156   if (duration == 0) {
        return -1;
176   }
177 …
178     payload[0] = GROUP_ID;
179     payload[1] = (uint8_t)(distance >> 8);
180     payload[2] = (uint8_t)(distance & 0xFF);
181
182     ________(APP_PORT, payload, sizeof(payload), 0);  // (5) LMIC function that schedules a
183 data transmission
184
        Serial.printf("[%s] Group %d sent a distance of %d cm to the gateway.\n",
233                   formatTimestamp(millis()).c_str(), GROUP_ID, distance);
234   }
235 …
236 }
237
238 void loop() {
      // All LoRaWAN scheduling is done by this function.
      ________();  // (6) Main function that runs the LMIC queue
    }
```

Gateway Monitor Source Code — ChirpStack MQTT (Orange Pi 4 PRO)

```
29  MQTT_PORT = 1883
30
31  # The ChirpStack uplink topic. The + means any application id and any device
32  # id, so one script watches every group at once.
33  UPLINK_TOPIC = ________  # (1) ChirpStack uplink topic, using + for every application and
34  device
35
36
37  def timestamp() -> str:
        """Returns the current wall-clock time in HH:MM:SS:MS format."""
46  …
47      return group, distance
48
49
50  def on_connect(client, userdata, flags, reason_code, properties=None) -> None:
51      client.________(UPLINK_TOPIC)  # (2) Method that registers a topic subscription
52      print(f"[{timestamp()}] Connected to the ChirpStack broker, waiting for uplinks...")
53
54
55  def on_message(client, userdata, msg) -> None:
56      received_at = timestamp()
57
58      event = ________(msg.payload)  # (3) Function that parses JSON text into a dict
59
60      # ChirpStack sends the payload as base64, so it has to be decoded first.
61      raw = base64.b64decode(event["data"])
62      group, distance = decode_payload(raw)
63
64      info = event.get("rxInfo", [{}])[0]
65      rssi = info.get("rssi", 0)
```

```
    snr = info.get("________", 0)  # (4) Name of the signal-to-noise field inside rxInfo

    print(f"[{received_at}] Group {group} | distance {distance} cm | RSSI: {rssi} dBm | SNR:
{snr} dB")
…
def main() -> None:
    client = mqtt.Client(mqtt.CallbackAPIVersion.VERSION2)
    client.on_connect = on_connect
    client.on_message = ________  # (5) Function that handles each incoming message

    client.connect(MQTT_HOST, MQTT_PORT, 60)
    client.________()  # (6) Method that runs the MQTT loop indefinitely


if __name__ == "__main__":
    main()
```

10. Record your Part B observations in the following table! The send time comes from the node's Serial Monitor, while the receive time, RSSI, and SNR come from the monitor program's output on the Orange Pi.

Trial no. …	Distance **[cm]**	RSSI **[dBm]**	SNR **[dB]**	Send time *(Node)* **[HH:MM:SS:MS]**	Receive time *(Gateway)* **[HH:MM:SS:MS]**	One-way delay **[ms]**
1.						
2.						
3.						
4.						
5.						
Average delay (ms)						

B. Analysis

1. Draw the communication flow implemented in the system you have worked on!

2. Next, a response-time experiment will be carried out for packets sent from the **Beacon** to your group's **Hunter**. Open the Hunter board's Serial Monitor; every packet it catches is printed with its packet number, raw RSSI, smoothed RSSI and SNR. The **send time** is carried by the packet itself from the

Beacon, while the **receive time** is stamped by the Hunter as the packet arrives, so both can be read from the same Serial Monitor. The same numbers also appear on the scoreboard at `http://192.168.4.1:8080`. Record:

- **FIVE (5)** rows of readings: packet number, RSSI, SNR, send time and receive time

Both times are recorded in **HH:MM:SS:MS** format. Compute the **one-way delay** by subtracting the send time from the receive time. This module has no round-trip column because a Hunter never transmits at all.

*Note: the Beacon and the Hunters all synchronise their clocks over NTP to the **same Orange Pi**, not to the Internet — the practicum hotspot has no route out. Because both ends start from one source of time, the difference between them is valid and may be subtracted, even though the Orange Pi's own absolute clock may be several seconds off the real time. If a board fails to synchronise, its timestamp is **not** a wall clock and that row must be left blank. RSSI and SNR can only be measured at the receiving end, so both come from the Hunter.*

Trial no. …	Packet no.	RSSI **[dBm]**	SNR **[dB]**	Send time *(Beacon)* **[HH:MM:SS:MS]**	Receive time *(Hunter)* **[HH:MM:SS:MS]**	One-way delay **[ms]**
1.						
2.						
3.						
4.						
5.						
Average one-way delay (ms)						

3. Finally, test how honest RSSI really is as a distance indicator. The scoreboard shows an **estimated distance in metres**, but that number is computed from RSSI with a *log-distance* model — it is not measured. Place the Hunter board at three distances you measure yourself with a tape, then record both the smoothed RSSI **and** the scoreboard's estimate for each position. Try one of them with a wall or your own body blocking the line of sight to the Beacon, and compare the difference.

Position	Real distance *(tape)* **[m]**	Smoothed RSSI **[dBm]**	Scoreboard estimate **[m]**	Obstacle? *(yes / no)*
Near				
Middle				
Far				

Note: that estimate in metres comes from

$$d = 10^{\frac{P_{1\mathrm{m}} - \mathrm{RSSI}}{10n}}$$

*where $P_{1\mathrm{m}}$ is the RSSI at one metre (the value you just calibrated) and n is the path-loss exponent, here 2.0 because the hunt is played on open ground; inside a partitioned building it is 2.5–3.5. The formula assumes the signal fades smoothly with distance — an assumption that collapses the moment a wall is in the way. That is why the scoreboard always shows a range rather than a single number. Notice too that the range **widens in proportion to the distance itself**: a ± 6 dB uncertainty means "10 m" is really 6–17 m, while "800 m" is really 460–1390 m. RSSI is therefore useful for guiding your steps once you are close, and very nearly useless as a statement of distance while you are still far away.*

4. Write your **analysis** of the system you experimented with earlier in the fill-in section below!

(Also give the **relationship/correlation** by applying the **communication methodology** you are studying in this module; it does not have to be complete!)

C. Suggestions and Feedback

1. Assistant

2. Laboratory